Runbook Finance Tracker: n8n + Postgres + Telegram

Catatan operasional untuk menyalakan, mematikan, membersihkan, menguji, dan merawat project. Diperbarui 11 November 2026.

1. Gambaran singkat

Project ini adalah bot keuangan di Telegram. Pesan masuk ke n8n lewat webhook, diproses workflow (termasuk AI Intent Router via OpenAI), lalu datanya disimpan di Postgres.

Komponen	Fungsi	Data disimpan di
Telegram (bot)	Antarmuka: kirim pesan, terima balasan	Server Telegram
Cloudflare Tunnel (cloudflared)	Jembatan agar Telegram bisa menembak n8n di laptop	Tidak menyimpan data
n8n	Menjalankan workflow. Editor di http://localhost:5678	Postgres dan volume n8n_data
Postgres	Data keuangan (accounts, budgets, savings, transactions), activity_log, fungsi SQL, dan tabel internal n8n (workflow, credential, riwayat eksekusi)	Volume postgres_data

File penting di folder project: `docker-compose.yml`, `start.sh`, `.env`, `init.sql` (tabel), `functions.sql` (fungsi SQL, dijalankan `start.sh` setiap kali start), `sanitize-workflow.sh`, serta hasil ekspor workflow `finance-tracker.json` (aman untuk GitHub) dan `finance-tracker.local.json` (berisi User ID asli, jangan di-commit).

Penting: n8n menyimpan datanya sendiri (workflow, credential, riwayat eksekusi) di database Postgres yang sama dengan tabel keuangan. Perintah apa pun yang menghapus seluruh database juga menghapus workflow-mu.

2. Lima konsep yang perlu dipahami

1. **Container itu sekali pakai, volume yang menyimpan data.** Container boleh dihentikan, dihapus, atau dibuat ulang kapan saja. Data aman selama volume `postgres_data` dan `n8n_data` tidak dihapus.
2. **URL tunnel berganti setiap tunnel dinyalakan.** n8n harus tahu URL terbarunya (`WEBHOOK_URL`) supaya Telegram menembak alamat yang benar. Karena itu, selalu nyalakan project lewat `start.sh`, bukan `docker compose up -d` atau `docker compose start` biasa.
3. **N8N_ENCRYPTION_KEY tidak boleh hilang atau diganti.** Key ini mengenkripsi credential di n8n. Simpan salinannya di password manager. Kalau berubah, workflow tetap ada tapi credential tidak bisa dibaca lagi.
4. **Save dan Publish itu berbeda.** Di n8n 2.x, Save hanya menyimpan draft. Publish menjadikan versi itu yang dipakai pesan Telegram. Kalau kamu mengubah node tapi tidak Publish, pesan asli

tetap memakai versi lama. Klik Execute di editor memakai isi editor, jadi tes bisa lolos di editor tetapi berperilaku lain dari Telegram.

5. **URL tunnel baru butuh Unpublish lalu Publish.** `start.sh` memberi n8n URL tunnel yang baru, tetapi Telegram baru tahu alamat itu saat workflow mendaftarkan webhook-nya, yaitu ketika workflow di-publish. Setelah setiap `bash start.sh`, unpublish workflow lalu publish lagi (bagian 4).

3. Melanjutkan project (setelah laptop dimatikan)

1. Nyalakan Docker Desktop, tunggu sampai statusnya running.
2. Buka folder project di File Explorer, klik kanan di area kosong, pilih Open Git Bash here.
3. Jalankan:

```
bash start.sh
```

Tunggu sampai muncul tulisan `Done.` dan URL `https://....trycloudflare.com`. Skrip ini juga menerapkan `functions.sql` (aman diulang, tidak mengubah datamu) dan berhenti dengan pesan error kalau gagal.

4. Cek semua container hidup:

```
docker compose ps
```

Harus ada tiga container: budget-postgres (healthy), budget-n8n, dan budget-cloudflared.

5. Buka http://localhost:5678 dan login. Kalau editor sudah terbuka di browser sebelum `start.sh`, refresh halamannya.
6. Unpublish lalu Publish workflow finance-tracker (bagian 4). Langkah inilah yang mendaftarkan URL tunnel baru ke Telegram. Pastikan statusnya kembali Published.
7. Tes: kirim `/start` atau satu pesan ke bot di Telegram, lalu lihat tab Executions di n8n.
8. Kalau bot diam: cek webhook dengan `getWebhookInfo` (bagian 4), Unpublish dan Publish lagi, lalu cek log dengan `docker compose logs n8n`.

> *Setelah laptop restart, container postgres dan n8n bisa menyala sendiri (karena restart: unless-stopped) dengan URL tunnel yang sudah lama. Tetap jalankan bash start.sh: skrip ini menyalakan tunnel baru lalu membuat ulang n8n dengan URL yang benar.*

4. Setiap kali start.sh: Unpublish lalu Publish (webhook tunnel)

Cloudflare quick tunnel yang gratis mendapat URL acak baru (`https://xxxx.trycloudflare.com`) setiap kali dinyalakan. `start.sh` meneruskan URL itu ke n8n sebagai `WEBHOOK_URL`, tetapi Telegram tidak membacanya dari sana: n8n yang memberi tahu Telegram alamatnya (lewat `setWebhook`) pada saat workflow di-publish. Workflow yang tetap berstatus published melewati restart bisa membuat Telegram tetap menunjuk ke URL lama yang sudah mati, sehingga bot terlihat diam padahal semuanya hijau. Unpublish lalu Publish memaksa n8n mendaftarkan URL yang baru. Di setup ini, anggap langkah itu wajib setiap kali tunnel baru menyala.

Langkah-langkahnya:

1. Jalankan `bash start.sh` dan tunggu sampai muncul `Done.` dan URL tunnel yang baru.
2. Buka http://localhost:5678 (refresh kalau sudah terbuka) lalu buka workflow finance-tracker.

3. Klik Unpublish dan konfirmasi. Workflow kembali menjadi draft dan bot berhenti membalas; itu wajar.
4. Klik Publish. Kalau muncul error seperti `Bad Request: bad webhook ... Failed to resolve host`, alamat tunnel baru belum bisa dijangkau: tunggu 20 sampai 30 detik lalu Publish lagi.
5. Cek webhook yang terdaftar (di bawah), lalu kirim satu pesan tes, misalnya `/start`.

Cek webhook yang terdaftar. Ganti `<TOKEN>` dengan token bot-mu. Jangan pernah menempelkan token asli di chat, screenshot, atau commit.

```
curl "https://api.telegram.org/bot<TOKEN>/getWebhookInfo"
```

Baca tiga field di jawaban JSON-nya: `url` harus diawali URL tunnel yang dicetak `start.sh`, `pending_update_count` sebaiknya 0 atau cepat turun, dan `last_error_message` sebaiknya tidak ada.

Yang terlihat di getWebhookInfo	Artinya dan solusinya
`url` berisi alamat trycloudflare yang lama	Webhook belum didaftarkan ulang. Unpublish, Publish lagi, lalu jalankan getWebhookInfo sekali lagi.
`url` kosong	Tidak ada webhook terdaftar: workflow belum di-publish. Publish.
`last_error_message`: Wrong response from the webhook: 530 (atau halaman Cloudflare 1033)	Telegram memanggil tunnel yang sudah tidak tersambung (URL lama). Solusinya sama: Unpublish lalu Publish.
`last_error_message`: 404	n8n terjangkau, tetapi tidak ada workflow published yang memiliki path itu (workflow di-unpublish, atau Telegram Trigger diubah). Publish, atau periksa node Telegram Trigger.
`last_error_message` soal timeout	Tunnel lambat atau mati. Cek `docker compose logs cloudflared`, lalu jalankan ulang `bash start.sh`.
`pending_update_count` terus naik	Pesan menunggu karena Telegram belum bisa mengirimkannya. Perbaiki penyebab di atas; pesan lama akan masuk (bisa terlambat) setelah webhook berfungsi.

Jangan menyalakan atau me-restart n8n dengan `docker compose up` atau `docker compose restart n8n` saja. `WEBHOOK_URL` lalu jatuh ke `http://localhost:5678`, alamat yang tidak bisa dijangkau Telegram (butuh HTTPS publik). Selalu lewat `bash start.sh`.

> *Aturan praktis: tunnel baru berarti URL baru berarti Unpublish lalu Publish. Mengubah node berarti Save lalu Publish (tanpa perlu Unpublish). Mematikan project dengan docker compose stop tidak perlu Unpublish.*

5. Mematikan project (selesai kerja)

1. Kalau hari ini kamu mengubah workflow: export dari n8n (menu titik tiga, Download), simpan di repo, lalu commit.
2. Di folder project, jalankan:

```
docker compose stop
```

Ini menghentikan semua container tanpa menghapus apa pun. Data, workflow, dan credential aman.

3. Cek dengan `docker compose ps -a`. Semua container harus berstatus Exited.
4. Setelah itu aman untuk menutup Docker Desktop atau mematikan laptop.

Catatan: mematikan laptop tanpa `docker compose stop` biasanya tidak merusak data (Postgres punya mekanisme pemulihan), tapi menghentikan dulu lebih aman. Selama project mati, bot tidak akan membalas; itu normal.

6. stop, down, dan down -v: apa bedanya

Perintah	Container	Volume (data)	Pakai untuk
`docker compose stop`	Dihentikan, tetap ada	Aman	Menutup sesi kerja harian
`docker compose down`	Dihapus (beserta network)	Aman	Membersihkan setelah ubah docker-compose.yml, atau saat ada container yang macet
`docker compose down -v`	Dihapus	**DIHAPUS**	Reset total, mulai dari nol

Kapan pakai `docker compose down` (data tetap aman, nyalakan lagi dengan `bash start.sh`):

- Setelah mengubah struktur docker-compose.yml, misalnya mengganti nama service, port, atau menghapus service.
- Saat muncul error `container name already in use` atau ada container yang nyangkut.
- Kalau ingin bersih-bersih container dan network tanpa menyentuh data.

Kapan pakai `docker compose down -v` (semua data hilang, backup dulu):

- Mengganti `POSTGRES_USER`, `POSTGRES_PASSWORD`, atau `POSTGRES_DB` padahal volume sudah ada. Postgres hanya membaca nilai ini saat volume pertama kali dibuat, jadi perubahan di `.env` tidak berpengaruh sebelum volume dihapus.
- Mengubah `init.sql` dan ingin tabel dibuat ulang dari nol (`init.sql` hanya berjalan saat volume masih kosong). Untuk tambahan kecil seperti satu tabel baru, bagian 7 sudah cukup.
- Menguji instalasi bersih, misalnya memastikan dokumentasi ini bisa diikuti dari awal.
- Data rusak dan lebih mudah memulai ulang.

Sebelum `down -v`: backup database (bagian 9) dan export workflow. Setelah `down -v`: jalankan `bash start.sh`, lalu import workflow, buat ulang credential di n8n, dan Publish (bagian 4).

Ingat: `down -v` juga menghapus workflow dan credential n8n, karena n8n menyimpannya di database Postgres yang sama. Kalau tujuanmu hanya mengosongkan data tes, pakai bagian 7.

7. Membersihkan data tes tanpa reset penuh

Sebelum menjalankan test plan, biasanya yang kamu butuhkan hanya tabel keuangan yang kosong, bukan environment Docker yang baru. Reset penuh (`down -v`) juga menghapus workflow, credential, dan pendaftaran webhook, sehingga kamu harus import, menyambungkan credential, dan Publish dari awal.

Situasi	Yang dipakai
Data tes dari putaran sebelumnya masih ada; skema dan workflow baik-baik saja	TRUNCATE tabel bot (di bawah). Workflow, credential, dan index tetap utuh.
Container nyangkut atau nama container sudah dipakai	`docker compose down`, lalu `bash start.sh`. Data tetap aman.
Kamu mengubah `init.sql` (kolom, constraint, index) atau menambah tabel	Terapkan perubahannya manual dengan ALTER TABLE atau CREATE, atau jalankan `init.sql` lagi (di bawah, aman diulang). `down -v` hanya jika kamu rela kehilangan semuanya, karena `init.sql` hanya berjalan di volume kosong.
Kamu mengganti `POSTGRES_USER`, `POSTGRES_PASSWORD`, atau `POSTGRES_DB`	`down -v` (backup dulu). Postgres hanya membaca nilai itu saat volume pertama dibuat.
Ingin menguji instalasi bersih dari nol	`down -v`, lalu ikuti bagian 3 dan import workflow.
Credential tidak bisa didekripsi	Kembalikan `N8N_ENCRYPTION_KEY` yang benar dulu. Reset hanya sebagai langkah terakhir.

Mengosongkan data bot saja: masuk ke database,

```
docker compose exec postgres sh -c 'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB"'
```

lalu jalankan:

```
TRUNCATE transactions, savings, budgets, accounts, activity_log RESTART IDENTITY CASCADE;
```

- **Yang dilakukan:** menghapus semua baris di lima tabel bot (accounts, budgets, savings, transactions, activity_log) dan mengembalikan penghitung ID ke 1, jadi ID di checkpoint test plan cocok. Index dan constraint tetap ada.
- **Yang tetap aman:** workflow, credential, riwayat eksekusi, serta status tunnel dan Published.
- **Yang tidak boleh disentuh:** tabel milik n8n sendiri (misalnya `workflow_entity`, `credentials_entity`, `execution_entity`). Jangan TRUNCATE atau DROP tabel itu, dan jangan jalankan `DROP SCHEMA public`.

Verifikasi: semua angka harus 0.

```
SELECT (SELECT count(*) FROM accounts) AS accounts, (SELECT count(*) FROM budgets) AS budgets, (SELECT count(*) FROM savings) AS savings, (SELECT count(*) FROM transactions) AS transactions, (SELECT count(*) FROM activity_log) AS activity_log;
```

Cek skema masih sesuai `init.sql` (perlu hanya kalau ada yang janggal):

```
\dt
SELECT indexname FROM pg_indexes WHERE tablename IN ('accounts','budgets','savings','transactions','activity_log') ORDER BY 1;
```

Harus ada lima tabel bot di antara tabel n8n, serta index antara lain `uq_accounts_name`, `uq_budgets_category`, `uq_savings_name`, `uq_transactions_source_key`, dan dua index `idx_activity_log_*`. Kalau ada yang kurang, terapkan seperti penjelasan di bawah. Membuat ulang database adalah pilihan paling akhir. Backup dulu (bagian 9) kalau datanya penting.

Menambah tabel atau index ke database yang sudah ada (misalnya `activity_log`, kalau databasemu dibuat sebelum tabel itu ada): `init.sql` hanya berjalan di volume kosong, tetapi semua perintah di dalamnya memakai `IF NOT EXISTS`, jadi seluruh file boleh dijalankan lagi tanpa kehilangan data. Jalankan dari Git Bash di folder project:

```
docker compose exec -T postgres sh -c 'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" -v ON_ERROR_STOP=1' < init.sql
```

Checklist sebelum memulai test plan:

1. `bash start.sh`, lalu tunggu URL tunnel baru.
2. Unpublish lalu Publish workflow, dan cek `getWebhookInfo` (bagian 4).
3. Kosongkan tabel bot dengan TRUNCATE dan pastikan semua angka 0.
4. Kirim `halo` ke bot (smoke test) dan pastikan ada balasan sapaan tetap. Perhatikan bahwa ini membuat baris pertama di `activity_log`.
5. Mulai test plan dari baris pertama.

8. Perintah sehari-hari

Tujuan	Perintah
Lihat status container	`docker compose ps -a`
Lihat log n8n (Ctrl+C untuk keluar)	`docker compose logs -f n8n`
Lihat log tunnel	`docker compose logs cloudflared`
Validasi docker-compose.yml	`docker compose config`
Masuk ke database	`docker compose exec postgres sh -c 'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB"'`
Terapkan `functions.sql` setelah diedit	`bash start.sh` (file ini dijalankan setiap start)
Hentikan tunnel saja (saat tidak demo)	`docker compose stop cloudflared`
Update n8n ke versi terbaru	`docker compose pull n8n`, lalu `bash start.sh` (backup dulu). Image belum di-pin, jadi sebaiknya pin versi tag di `docker-compose.yml`. Migration report n8n 3.0 hanya informasi: baca dulu, lalu tentukan kapan upgrade
Cek webhook Telegram (ganti <TOKEN>)	`curl "https://api.telegram.org/bot<TOKEN>/getWebhookInfo"`
Restart n8n dengan benar	`bash start.sh` (bukan `docker compose restart n8n`), lalu Unpublish dan Publish
Kosongkan data bot saja (di dalam psql)	`TRUNCATE transactions, savings, budgets, accounts, activity_log RESTART IDENTITY CASCADE;`
Jumlah baris kelima tabel bot (di dalam psql)	`SELECT (SELECT count(*) FROM accounts) AS accounts, (SELECT count(*) FROM budgets) AS budgets, (SELECT count(*) FROM savings) AS savings, (SELECT count(*) FROM transactions) AS transactions, (SELECT count(*) FROM activity_log) AS activity_log;`

Tujuan	Perintah
Lihat aktivitas bot terbaru (di dalam psql)	`SELECT log_id, created_at, status, intents, left(message, 60) AS message FROM activity_log ORDER BY log_id DESC LIMIT 20;`

Di dalam psql: `\dt` untuk melihat tabel, `SELECT * FROM accounts;` untuk melihat isi tabel, `\q` untuk keluar.

9. Backup, restore, dan membagikan workflow

Backup seluruh database (aman dijalankan dari PowerShell maupun Git Bash):

```
mkdir backups
docker compose exec -T postgres sh -c 'pg_dump --clean --if-exists -U "$POSTGRES_USER" -d "$POSTGRES_DB" -f /tmp/backup.sql'
docker compose cp postgres:/tmp/backup.sql ./backups/backup.sql
```

Cara ini sengaja tidak memakai > di PowerShell, karena PowerShell versi lama menyimpan hasil redirect dalam UTF-16 sehingga file SQL-nya rusak saat di-restore.

Hanya data keuangan (accounts, budgets, savings, transactions sebagai perintah INSERT, berguna untuk menyimpan atau memindahkan datamu tanpa tabel n8n):

```
docker compose exec -T postgres sh -c 'pg_dump -U "$POSTGRES_USER" -d "$POSTGRES_DB" --data-only --column-inserts -t accounts -t budgets -t savings -t transactions -f /tmp/finance-data.sql'
docker compose cp postgres:/tmp/finance-data.sql ./backups/finance-data.sql
```

Restore backup lengkap (n8n dihentikan dulu supaya tidak menulis saat restore):

```
docker compose stop n8n cloudflared
docker compose cp ./backups/backup.sql postgres:/tmp/backup.sql
docker compose exec -T postgres sh -c 'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" -f /tmp/backup.sql'
bash start.sh
```

Backup lengkap berisi seluruh database: data keuanganmu, activity log (teks pesan dan balasan), plus workflow dan credential n8n (credential terenkripsi dengan `N8N_ENCRYPTION_KEY`). Simpan privat. Setelah restore, `bash start.sh` dijalankan lagi, jadi lakukan juga Unpublish dan Publish (bagian 4).

Export workflow: menu titik tiga di n8n, Download. Simpan di repo GitHub (JSON-nya tidak berisi secret). Jangan meng-commit `.env` atau folder `backups/`, karena file backup berisi data keuanganmu.

Membagikan workflow di GitHub. Hasil ekspor memuat User ID Telegram aslimu di node Telegram Trigger. Sebelum commit:

1. Simpan hasil ekspor sebagai `finance-tracker.local.json` (nama ini diabaikan git lewat `.gitignore`).
2. Jalankan:

```
bash sanitize-workflow.sh finance-tracker.local.json
```

3. Skrip ini membuat `finance-tracker.json` dengan User ID diganti `YOUR_TELEGRAM_USER_ID` dan `instanceId` dikosongkan, lalu berhenti dengan error kalau User ID, token bot, API key `sk-`, atau URL `trycloudflare.com` masih ada. Commit hanya file ini, ditambah `.env.example` (tanpa nilai asli).
4. `.gitignore` sudah mengabaikan `.env`, `*.local.json`, folder `backups/`, dan `*.sql.gz`. Jangan paksa menambahkannya.

10. Apa yang dimengerti bot (perintah dan kata kunci)

Bot punya dua perintah dan memahami selebihnya sebagai bahasa biasa. Kedua perintah membalas dengan teks tetap, jadi tidak memanggil AI dan tidak menyentuh database.

Perintah	Fungsinya
`/start`	Pengenalan: cara kerja bot dalam tiga langkah dan aturan singkat (jenis pesan, akun lebih dulu dan budget opsional, batas nominal, tanggal, pesan gabungan, tidak ada edit atau batal).
`/help`	Panduan penggunaan: format dan kata kunci untuk Financial Setup, catat transaksi, dan Financial Insights. Kata `bantuan`, `panduan`, dan `cara pakai` melakukan hal yang sama.

Di BotFather, kirim `/setcommands` lalu tempel daftar ini supaya Telegram menampilkan perintahnya di menu:

```
start - Cara kerja bot dan aturan singkat
help - Format dan kata kunci tiap fitur
```

Cara pesan dikelompokkan (AI Intent Router membaca kata kuncinya):

Jenis	Kata pemicu	Contoh
Catat transaksi	makan, beli, bayar, gaji masuk, dapat, terima, bonus, untung, transfer, kirim, pindah, top up, tabung, setor	`makan siang 50rb pakai BCA`, `transfer 500rb dari BCA ke OVO`
Financial setup	buat, bikin, ubah, ganti, set, tambah, kurangi, hapus + akun, budget, atau savings	`buat akun BCA tipe bank saldo awal 5jt`, `ubah deadline savings rumah jadi juni 2028`
Insight	pertanyaan soal saldo, budget, savings, atau transaksi, atau pengandaian (`kalau aku beli ...`)	`sisa uangku berapa?`, `kalau aku beli laptop 10jt gimana?`

- Akun butuh nama, tipe (`cash`, `bank`, `e-wallet`, `credit card`, `loan`, `other`), dan saldo awal. Budget butuh nama dan limit bulanan. Savings butuh nama dan target; deadline opsional.

- Yang bisa diubah kemudian: akun (nama, tipe, saldo), budget (nama, limit), savings (nama, target, deadline), dan masing-masing bisa dihapus. Hapus massal ("hapus semua akun") diblokir.
- Satuan nominal: `rb` / `ribu` / `k`, `jt` / `juta`, `M` / `miliar`. Satu transaksi harus antara Rp1 dan Rp100 miliar.
- **Budget bersifat opsional.** Pengeluaran tanpa budget yang cocok tetap dicatat (kategori dari AI, atau `other`) dan balasannya menyebut budget belum dibuat. Hanya pengeluaran yang menyebut budget yang tidak ada yang ditolak.
- **Setup butuh kata buat.** Membuat akun, budget, atau savings butuh kata seperti `buat`, `bikin`, `set`, `atur`, atau `tambah akun`. Tanpa kata itu bot membalas pesan fallback dan tidak membuat apa pun (misalnya `BCA tipe bank saldo awal 10jt`). `tambahin 10k di akun BCA` menyesuaikan saldo, sedangkan `tambahin akun baru ...` membuat akun. Pertanyaan saran seperti `enaknya nama budget apa ya?` dijawab sebagai insight dan tidak memicu setup.
- **Kategori income.** AI membaca sumbernya dari pesan (`gaji`, `bonus`, `freelance`, `komisi`, `dividen`, `cashback`, `jual ...`). Kalau AI mengosongkan kategori, daftar kata kecil di node Normalize Transactions mengisinya dari pesan. Kalau tidak ada kata sumber, income dicatat sebagai `other` dan balasan meminta kamu menyebut sumbernya.
- **Memilih budget dengan sengaja.** Budget dianggap dipilih eksplisit hanya lewat kata `kategori <nama>` atau `masuk ke budget <nama>`. Kalau budget itu tidak ada, pengeluaran ditolak. Menulis `budget coffee` saja tidak cukup, jadi `beli kopi 20rb pakai BCA budget coffee` dicatat ke food lewat kata `kopi`.
- **Tabel dan jawaban cara pakai.** Permintaan rincian (`rincian`) dijawab dengan tabel yang dicetak aplikasi dari baris database, dalam blok kode monospace, dan bisa difilter per kategori (`pengeluaran shopping`). Pertanyaan cara memakai bot (`cara hapus akun`) dijawab dari panduan bawaan dan ditutup dengan arahan ke `/help`. `halo`, `hi`, dan `kamu bisa apa?` dijawab dengan balasan tetap.
- Satu pesan boleh berisi beberapa hal sekaligus (sampai 10 transaksi, ditambah perintah setup dan pertanyaan). Balasannya datang sebagai pesan terpisah per jenis.

11. Tanggal, bulan, dan zona waktu

- Container berjalan di zona waktu Asia/Jakarta (`TZ`, `PGTZ`, `GENERIC_TIMEZONE`), jadi "hari ini" dan pergantian bulan mengikuti WIB: bulan berganti pukul 00.00 WIB.
- **Setiap transaksi mendapat `transaction_date`.** Isinya tanggal yang kamu tulis di pesan, tanggal relatif yang dihitung kode dari tanggal hari ini (`kemarin`, `2 hari lalu`, `2 days ago`, `seminggu lalu`), atau hari ini kalau tidak ada tanggal.
- Pemakaian budget, peringatan melebihi limit, total pemasukan dan tabungan bulanan, serta insight semuanya mengikuti `transaction_date`. Transaksi bertanggal 20 Oktober masuk hitungan Oktober walaupun dicatat di bulan November. Hanya baris lama tanpa tanggal yang memakai `created_at`.
- **Maksimal 60 hari ke belakang.** Tanggal yang lebih lama, frasa yang tidak jelas seperti `bulan lalu` atau `2 bulan lalu`, dan tanggal di masa depan (`besok`, `lusa`, `3 hari lagi`, `tomorrow`, `in 3 days`) ditolak dengan pesan dan tidak ada yang tercatat (saldo tidak berubah). Tanpa pengecekan ini, pesan seperti `makan siang 50rb pakai BCA besok` akan tercatat dengan tanggal hari ini. Pada pesan berisi beberapa transaksi, hanya transaksi yang bermasalah yang ditolak.

- Belum dikenali: nama hari (`senin lalu`), `tadi malam`, dan tanggal tanpa tahun. Pesan dengan dua tanggal relatif yang berbeda dicatat dengan tanggal hari ini.
- Tanggal di perintah setup ("tahun depan") dihitung parser dari tanggal hari ini. Untuk deadline savings, tahun saja berarti 31 Desember, dan tanggal yang tidak ada (31 Februari) dijawab dengan pesan tanggal tidak valid.
- Jawaban insight ditulis AI dari angka database, jadi cek jawaban berbasis bulan dengan SQL kalau penting.
- Baris lama yang dicatat sebelum perilaku ini bisa punya `transaction_date` kosong. Perintah ini mengisinya dari `created_at` (aman dijalankan sekali):

```
UPDATE transactions SET transaction_date = created_at WHERE transaction_date
IS NULL;
```

12. Memperbaiki data yang salah dicatat

Bot tidak punya edit atau undo untuk transaksi. Perbaiki langsung di Postgres, dan selalu balik efek saldonya bersama barisnya dalam satu transaksi database (`BEGIN` … `COMMIT`). Kesalahan setup (nilai akun, budget, atau savings) cukup dikoreksi lewat bot, misalnya `set saldo BCA 2jt` atau `ubah budget food jadi 700rb`.

Cari dulu ID transaksinya (masuk ke psql seperti di bagian 8):

```
SELECT transaction_id, transaction_date, transaction_type, amount, category,
notes, from_account_id, to_account_id, savings_id FROM transactions ORDER BY
transaction_id DESC LIMIT 10;
```

Lalu balik sesuai jenisnya. Ganti `42` dengan `transaction_id` yang sebenarnya. Setelah itu, kirim lagi pesan yang benar ke bot kalau transaksinya masih mau dicatat.

Jenis	Yang dibalik
expenses	Kembalikan nominal ke `from_account_id`
income	Kurangi nominal dari `to_account_id`
transfer	Kembalikan ke `from_account_id` dan kurangi dari `to_account_id`
savings	Kembalikan ke `from_account_id` dan kurangi `current_amount` savings-nya

Pengeluaran:

```
BEGIN;
UPDATE accounts a SET balance = a.balance + t.amount, updated_at = NOW()
  FROM transactions t WHERE t.transaction_id = 42 AND a.account_id =
t.from_account_id;
DELETE FROM transactions WHERE transaction_id = 42;
COMMIT;
```

Pemasukan:

```
BEGIN;
UPDATE accounts a SET balance = a.balance - t.amount, updated_at = NOW()
  FROM transactions t WHERE t.transaction_id = 42 AND a.account_id =
```

```
t.to_account_id;
DELETE FROM transactions WHERE transaction_id = 42;
COMMIT;
```

Transfer:

```
BEGIN;
UPDATE accounts a SET balance = a.balance + t.amount, updated_at = NOW()
  FROM transactions t WHERE t.transaction_id = 42 AND a.account_id =
t.from_account_id;
UPDATE accounts a SET balance = a.balance - t.amount, updated_at = NOW()
  FROM transactions t WHERE t.transaction_id = 42 AND a.account_id =
t.to_account_id;
DELETE FROM transactions WHERE transaction_id = 42;
COMMIT;
```

Setoran savings:

```
BEGIN;
UPDATE accounts a SET balance = a.balance + t.amount, updated_at = NOW()
  FROM transactions t WHERE t.transaction_id = 42 AND a.account_id =
t.from_account_id;
UPDATE savings s SET current_amount = s.current_amount - t.amount, updated_at
= NOW()
  FROM transactions t WHERE t.transaction_id = 42 AND s.savings_id =
t.savings_id;
DELETE FROM transactions WHERE transaction_id = 42;
COMMIT;
```

Kalau hanya tanggalnya yang salah (pemakaian budget mengikuti tanggal, jadi ini memindahkan transaksi ke bulan yang benar):

```
UPDATE transactions SET transaction_date = '2026-11-05' WHERE transaction_id
= 42;
```

Setelahnya cek saldo dengan `SELECT account_name, balance FROM accounts;` dan tanya bot `sisa uangku berapa?`. Keduanya harus sama.

13. Membaca activity log

Setiap balasan bot juga ditulis ke `activity_log`: ID update Telegram, ID chat, pesanmu, intent yang terdeteksi, status, dan teks balasan. Status: `succeeded`, `rejected` (ada yang ditolak, misalnya akun tidak dikenal), `mixed` (sebagian diterima, sebagian ditolak), dan `error`. Kalau penyimpanan log itu sendiri gagal, bot tetap membalas.

```
SELECT log_id, created_at, status, intents, left(message, 60) AS message FROM
activity_log ORDER BY log_id DESC LIMIT 20;
```

Hitung per status:

```
SELECT status, count(*) FROM activity_log GROUP BY status ORDER BY 2 DESC;
```

Cari tahu kenapa sesuatu tidak tercatat (apa yang kamu kirim dan jawaban bot):

```
SELECT log_id, created_at, message, reply FROM activity_log WHERE status IN
('rejected','mixed','error') ORDER BY log_id DESC LIMIT 20;
```

Log berisi teks pesan dan nominalmu. Log ini tetap di database lokal, jadi jangan meng-commit dump database.

14. Test plan dan pelajaran dari pengujian

Bot diuji dalam tiga putaran. Setiap test plan berisi pesan yang dikirim, hasil yang diharapkan, dan tempat untuk hasil sebenarnya. Lembar hasil yang sudah disesuaikan mencatat apa yang terjadi dan apa yang diubah sesudahnya. Test plan adalah panduan. Kolom harapannya ditulis sebelum perbaikan terakhir, jadi lembar hasil adalah catatan yang paling baru.

Test plan	Cakupan	Lolos di putaran pertama	Diperbaiki sesudahnya
1st: Financial Setup	Akun, budget, savings: buat, ubah, sesuaikan, hapus, validasi, penanganan tanggal	78 dari 85	7
2nd: Transaksi dan Insight	Income, expense, transfer, savings, kasus negatif, insight, multi-intent, kasus tepi tambahan	71 dari 80	9
3rd: Versi GitHub	Perintah tetap, guard kata buat, budget opsional, kasus negatif, tabel, pertanyaan cara pakai dan saran, pesan gabungan	56 dari 57	1 (tanggal masa depan), ditambah 1 penguatan (sumber income)

Putaran ke-3 lolos di semua bagian kecuali satu: `besok` tercatat dengan tanggal hari ini, dan sudah diperbaiki di Normalize Transactions (bagian 10). Daftar kata sumber income ditambahkan setelah `freelance 1,5jt ke OVO` tercatat sebagai `other`. Ada dua catatan kecil: jawaban saran bisa mengulang nama budget, dan susunan kata status budget bisa bervariasi. Keduanya tidak mengubah angka apa pun.

Hal yang perlu diingat saat menjalankan test plan:

- Jalankan fase secara berurutan, masing-masing di tabel kosong (bagian 7). Transaksi butuh akun, budget, dan savings dari fase setup.
- Pakai nama budget bahasa Inggris (`food`, `transport`, `entertainment`) supaya nama kategori dari parser cocok dengan budget-mu. Kategori yang tidak cocok dengan budget mana pun ditolak dengan saran.
- Cek tiga tempat untuk setiap pesan: balasan di Telegram, tab Executions di n8n, dan database.
- Pengeluaran butuh akun sumber yang disebut, dan setiap transaksi dalam pesan multi-transaksi divalidasi sendiri-sendiri.
- Pertanyaan pengandaian tidak boleh membuat transaksi. Pastikan dengan `SELECT count(*) FROM transactions;` sebelum dan sesudah.
- Setoran savings tanpa nama tujuan masuk ke savings bernama `general`, jadi buat dulu.

- Cek satuan dan ukuran: `1M` adalah satu miliar rupiah. Income di atas Rp100 miliar (`uang bonus 1T masuk ke BCA`) ditolak karena ukurannya, sedangkan pengeluaran sangat besar (`beli rumah 5T pakai BCA`) ditolak karena saldo tidak cukup.
- Input campuran (Inggris, Indonesia, Mandarin dalam satu pesan) masih bisa meleset. Catat hasilnya sebagai temuan dan, kalau perlu, perbaiki di prompt.

15. Troubleshooting

Gejala	Penyebab dan solusi
`bash start.sh` error `\r` atau `bad interpreter`	Akhir baris Windows (CRLF). Jalankan `sed -i 's/\r$//' start.sh`
`Cannot connect to the Docker daemon` atau `docker: command not found`	Docker Desktop belum jalan
Gagal mendapatkan URL tunnel	Cek `docker compose logs cloudflared` dan koneksi internet
`start.sh` bilang functions.sql gagal diterapkan	Baca error SQL yang tercetak di atas pesan itu, perbaiki `functions.sql`, lalu jalankan `bash start.sh` lagi. File ini tidak mengubah datamu.
Bot diam, tab Executions kosong	Unpublish lalu Publish workflow (bagian 4) dan cek getWebhookInfo. Pastikan kamu menyalakan lewat `start.sh` (URL terbaru) dan User ID di Telegram Trigger sudah benar
Eksekusi merah di node Postgres	Credential Postgres: host harus `postgres`, bukan `localhost`
Eksekusi merah di node Log Activity, atau `relation "activity_log" does not exist`	Tabelnya belum ada (database dibuat sebelum tabel ini ditambahkan). Jalankan `init.sql` lagi seperti di bagian 7. Bot tetap membalas walaupun log gagal.
Eksekusi merah di node OpenAI	API key salah atau kuota habis
`container name already in use`	Jalankan `docker compose down`, lalu `bash start.sh`
Warning bahwa sebuah `variable is not set`	Ada karakter `$` di nilai `.env`. Tulis `$$`, atau bungkus nilai dengan kutip tunggal
Credential n8n tidak bisa didekripsi	`N8N_ENCRYPTION_KEY` berbeda dari saat credential dibuat. Kembalikan key lama, atau buat ulang credential
Tabel tidak ada	Volume sudah ada sebelum `init.sql` di-mount. Jalankan `init.sql` manual (bagian 7), atau reset dengan `down -v`
Bot diam setelah `start.sh` baru, padahal workflow berstatus Published	URL tunnel baru belum terdaftar di Telegram. Unpublish, Publish, lalu cek getWebhookInfo (bagian 4)
Sudah mengubah node tetapi bot masih berperilaku lama	Perubahan baru berupa draft sampai di-Publish. Save lalu Publish
Berhasil di editor (Execute) tetapi tidak dari Telegram	Editor menjalankan draft, Telegram menjalankan versi yang di-publish. Publish

Gejala	Penyebab dan solusi
Telegram berhenti membalas setelah `docker compose up` atau restart di luar `start.sh`	n8n menyala dengan `WEBHOOK_URL=http://localhost:5678`. Jalankan `bash start.sh`, lalu Unpublish dan Publish
Error Telegram `can't parse entities`	Teks dengan karakter `_ * [` sampai ke Telegram sebagai Markdown. Node Send meng-escape-nya untuk balasan biasa; kalau kamu mengedit teks `/start` atau `/help`, hindari karakter itu atau pastikan backtick berpasangan
Tabel menampilkan backtick mentah di Telegram	Node Send tidak boleh meng-escape blok kode. Teks dipisah pada blok `` ` dan _ * [ `` hanya di-escape di luar blok
Eksekusi merah di node Telegram: `The connection to the server was closed unexpectedly` atau timeout	Biasanya error jaringan sesaat. Cek dari dalam container: `docker compose exec n8n wget -qO- --timeout=10 https://api.telegram.org`. Pertahankan Retry On Fail di setiap node Telegram, dan pastikan `NODE_OPTIONS=--dns-result-order=ipv4first` ada di bawah `environment:` service n8n. Setelah laptop sleep atau ganti jaringan, jalankan `bash start.sh`
Item yang dibuat tidak ada, atau bot bilang pesan setup belum lengkap	Pembuatan butuh kata buat (bagian 10). Tulis ulang dengan `buat` atau `bikin`
`start.sh` berhenti di functions.sql dan fungsi lama tetap berjalan	Ada error sintaks di `functions.sql` (yang sering: `ELSE IF` bukan `ELSIF`). Perbaiki lalu jalankan `bash start.sh` lagi
Nominal tercatat salah (`1M` jadi 1jt, atau `40k` dan `10k` dijumlahkan)	Parser AI salah membaca pesan. Tulis satu nominal per baris, atau balik lewat SQL (bagian 12). Guard yang mencocokkan nominal dengan pesan asli ada di roadmap
Bot menjawab "Aku belum paham pesan itu" untuk pesan yang valid	Router tidak menemukan intent. Tulis ulang dengan kata kunci di `/help`, atau cek eksekusi AI Intent Router
Income tercatat dengan kategori `other` padahal sudah menyebut sumber	Kata sumbernya mungkin belum ada di daftar. Tulis dengan bentuk umum (`gaji`, `bonus`, `freelance`, …) atau tambah baris di `INCOME_WORDS` pada Normalize Transactions (bagian 10)
Transaksi dengan `besok` tercatat dengan tanggal hari ini	Pengecekan tanggal masa depan tidak ada di Normalize Transactions. Pastikan `relativeDate()` mengembalikan `FUTURE` untuk `besok`, `lusa`, `tomorrow`, dan baris `problem` memetakannya ke `future`, lalu Publish ulang
Transaksi ditolak karena tanggalnya	Tanggal lebih lama dari 60 hari, tidak jelas (`bulan lalu`), atau di masa depan. Tulis tanggal lengkap dalam 60 hari terakhir, atau `kemarin` / `5 hari lalu` (bagian 11)

Gejala	Penyebab dan solusi
Insight menampilkan angka yang berbeda dari database	Angka berasal dari database, tetapi kalimatnya ditulis AI. Kalau ada catatan ⚠️, berarti ada angka yang tidak ada di data. Cek dengan SQL (bagian 8) dan catat sebagai temuan
Transaksi yang sudah tercatat ternyata salah	Tidak ada edit atau undo. Balik lewat SQL (bagian 12)

16. Aturan emas

- Jangan jalankan `docker compose down -v` kecuali memang mau menghapus semua data, termasuk workflow dan credential n8n.
- Selalu nyalakan lewat `bash start.sh`, lalu Unpublish dan Publish.
- Simpan `N8N_ENCRYPTION_KEY` di password manager.
- Jangan commit `.env` atau backup, dan jangan tempel token atau password di chat atau screenshot. Commit hanya `finance-tracker.json` yang sudah disanitize.
- Pasang spending limit di akun OpenAI.
- Matikan tunnel kalau tidak sedang demo, karena editor n8n ikut terjangkau lewat URL tunnel.
- Pastikan Telegram Trigger dibatasi ke User ID-mu.
- Jangan mengubah baris transaksi tanpa membalik saldonya dalam transaksi database yang sama.